FreeFlow 画布引擎与产品创新点横向分析

更新时间：2026-05-01

1. 分析范围与方法

这份分析不是只看 `canvas2d-core`，而是把 FreeFlow 当成一个完整产品来评估。
我实际检查的范围包括：

- 画布引擎与元素体系
- 结构化导入 / 复制粘贴 / 拖拽协议
- 结构化导出 / Word 导出 / 复制文本协议
- 本地画布文件体系与工作区管理
- Electron 桌面壳与窗口形态同步
- AI 助手工作区与 AI 镜像工作区
- 文件内容抽取、搜索、教程、启动恢复与偏好体系

结论基于两部分：

1. 项目代码静态审查
2. 主流产品官方资料的横向基线

对照参考主要包含：

- Miro
- FigJam
- Excalidraw
- AFFiNE
- Obsidian Canvas

说明：
我这里区分三类结论：

- **真正的产品创新点**：市面上较少见，或者至少组合方式非常少见
- **组合创新点**：单项不绝对独有，但组合在一起后形成明显差异
- **行业通用能力**：你有，但不能直接当成创新宣传

---

2. 先给结论：FreeFlow 本质上不是“普通白板”

如果一句话定义 FreeFlow：

> FreeFlow 不是单纯的无限画布，也不是单纯的 AI 聊天壳，而是一个把 **本地桌面工作台、结构化画布、办公文档导入导出、AI 镜像工作区、真实文件夹工作区** 融合在一起的桌面型工作系统。

这点非常关键。

和多数画布产品相比，FreeFlow 的差异不在“有个画布”，而在于它把以下几条链路打通了：

- 外部内容进入画布，不只是粘贴，而是**协议化、结构化、可降级、可回流**
- 画布中的内容，不只是显示，而是**可导出到 Word / Markdown / 纯文本 / TSV / XLSX 等办公流**
- 画布不是孤立文件，而是**映射真实本地文件夹的工作区**
- AI 不是外挂按钮，而是**独立工作区 + 外部 AI 页面镜像嵌入**
- 桌面壳不是普通 Electron 窗口，而是**透明、置顶、窗口 shape 同步、穿透控制**

这意味着 FreeFlow 更接近：

- “桌面工作白板”
- “本地 AI + 办公画布工作台”
- “结构化画布内容中台”

而不是 Miro / FigJam 那种纯协作白板，也不是 Excalidraw 那种轻量绘图白板，更不是 Obsidian Canvas 那种知识卡片连接板。

---

3. FreeFlow 最有价值的创新点

3.1 结构化导入架构，不是简单 paste handler

这是 FreeFlow 最硬核的架构创新之一。

很多画布产品对“粘贴 / 拖拽”的处理本质是：

- 能贴文本就贴文本
- 能贴图片就贴图片
- 能贴 HTML 就做个近似保留

而 FreeFlow 做的是一套独立的**结构化导入系统**：

- 有输入描述协议 `inputDescriptor`
- 有内容类型检测
- 有 parser registry
- 有 canonical document
- 有 renderer pipeline
- 有 fallback strategy
- 有 diagnostics / import log
- 有 rollout switch / kill switch
- 有 host flowback / host persistence

这不是 UI 小功能，这是完整的“内容接入层”。

代码证据：

- `public/src/engines/canvas2d-core/import/README.md`
- `public/src/engines/canvas2d-core/import/protocols/inputDescriptor.js`
- `public/src/engines/canvas2d-core/import/canonical/canonicalDocument.js`
- `public/src/engines/canvas2d-core/import/parsers/parserRegistry.js`
- `public/src/engines/canvas2d-core/import/renderers/rendererPipeline.js`
- `public/src/engines/canvas2d-core/import/diagnostics/importLogCollector.js`
- `public/src/engines/canvas2d-core/import/rollout/killSwitch.js`

这带来的创新意义是：

- FreeFlow 在架构上不是“把外部内容贴进画布”，而是“把外部内容翻译成内部内容协议”
- 后续任何新的输入源都可以挂进这条链路
- 可以兼顾兼容性、格式保留、回退、日志、灰度切换

这类设计，在普通白板工具里很少做到这么深。

3.2 “保留格式粘贴”不是表层，而是转译为自有元素体系

你举的这个例子是对的，而且确实是 FreeFlow 的核心创新点之一。

FreeFlow 的导入链路不是简单保留原始 HTML，而是尽量把外部内容转译成自己的元素体系，比如：

- 标题 / 段落 -> 文本元素
- Markdown -> 结构化文本 / 列表 / 表格 / 数学公式
- code -> 代码块元素
- table -> 表格元素
- math -> 数学公式元素
- image -> 图片元素
- file -> fileCard 元素

关键不是“能贴进去”，而是：

> **贴进来以后，不是异物，而是 FreeFlow 原生对象。**

代码证据：

- `public/src/engines/canvas2d-core/import/parsers/html/htmlParser.js`
- `public/src/engines/canvas2d-core/import/parsers/markdown/markdownParser.js`
- `public/src/engines/canvas2d-core/import/parsers/math/latexMathParser.js`
- `public/src/engines/canvas2d-core/import/renderers/text/textElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/table/tableElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/code/codeBlockElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/math/mathElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/file/fileCardElementBridge.js`

这比很多产品“贴成一个富文本框”更进一步。

它的价值在于：

- 后续还能继续编辑、搜索、导出、复制、再转换
- 不会变成黑箱对象
- 内部逻辑统一，后续自动排版、导出 Word、搜索索引才能成立

3.3 复制 / 拖拽 / 对外输出不是单格式，而是双层协议

FreeFlow 的复制输出不是只做 `text/plain` 或 `text/html`。

它实际上做了两层：

1. 内部 canonical / structured copy
2. 面向外部应用的 downgrade / compatibility output

也就是说，系统同时考虑：

- FreeFlow 内部再粘贴时，尽量保留结构和语义
- 粘贴到外部应用时，自动降级成稳定可识别的文本 / HTML / 文件路径

代码证据：

- `public/src/engines/canvas2d-core/import/protocols/canonicalFragmentCopy.js`
- `public/src/engines/canvas2d-core/import/protocols/copyDowngradeRules.js`
- `public/src/engines/canvas2d-core/import/protocols/externalCompatibilityOutput.js`
- `public/src/engines/canvas2d-core/import/host/hostFlowbackAdapter.js`

这个点的创新在于：

- FreeFlow 已经开始把“复制”视为一种内容协议，而不是一个浏览器 API 事件
- 这非常适合做跨应用办公流
- 也解释了为什么它能衍生出 Word 直通、PPT 直通、Markdown、纯文本等复制方式

3.4 画布内容直接进入 Word 办公流，这是非常强的差异化

这是 FreeFlow 最容易对外讲清楚、也最有商业价值的创新点之一。

很多白板产品：

- 强在协作
- 强在模板
- 强在讨论

但弱在“把整理结果变成正式文档”。

FreeFlow 这里做的是：

- 画布内容按视觉阅读顺序整理
- 转成导出 AST
- 再编译成 `.docx`
- 支持标题、段落、列表、引用、代码块、表格、数学公式、超链接、脚注等
- 还做了导出预览模型

代码证据：

- `public/src/engines/canvas2d-core/export/word/buildWordExportAst.js`
- `public/src/engines/canvas2d-core/export/word/buildWordExportPreviewModel.js`
- `electron/wordDocxCompiler.js`
- `electron/main.js`

这说明 FreeFlow 的目标不是“白板展示”，而是：

> **从画布整理，直接走向办公输出。**

这一点和 Miro / FigJam / Excalidraw 的核心价值主轴很不一样。

3.5 富文本元素支持多种“办公直通复制”

FreeFlow 不是只有导出文件，还在元素级复制上做了“办公直通”。

当前代码中已经有：

- 纯文本
- 富文本（Word 直通）
- 富文本（PPT直通）
- Markdown
- 对象链接

表格还有：

- Markdown 表格
- TSV（Excel 直达）

代码证据：

- `public/src/engines/canvas2d-core/export/copyExportProtocol.js`
- `public/src/engines/canvas2d-core/contextMenu/menuSchemaBuilders.js`

这很重要，因为它体现出 FreeFlow 不是只做“导出文件”，而是做“复制即流转”。

也就是说：

- 画布里的一个对象，本身就能成为 Office 流的一部分
- 用户不必总是先导出整个文件

这个产品思路很强，明显偏生产力工具，不是娱乐或轻量白板。

3.6 本地真实文件夹映射的画布工作区，而不是封闭的 App 内部列表

FreeFlow 的画布管理不是传统“应用内部数据库里的一堆板子”。

它正在走的是：

- 选择电脑里的真实文件夹
- 扫描其中的画布文件
- 直接在 UI 中做工作区管理
- 新建 / 打开 / 保存 / 另存为 / 重命名，全部映射真实文件

代码证据：

- `public/src/engines/canvas2d-core/ui/BoardWorkspaceDialog.jsx`
- `public/src/engines/canvas2d-core/createCanvas2DEngine.js`
- `src/backend/services/canvasBoardService.js`
- `src/backend/services/appStartupService.js`

这和 Obsidian Canvas 的“文件系统心智”有相似处，但 FreeFlow 这里更偏：

- 桌面应用工作区
- 白板工作区
- 本地项目空间

它的价值在于：

- 用户对文件位置有真实掌控
- 更容易接入个人资料夹、项目文件夹、长期积累
- 更适合桌面端的本地工作流

3.7 AI 镜像工作区，是 FreeFlow 非常罕见的产品创新

这是 FreeFlow 与绝大多数画布产品最不一样的点之一。

它不是普通“接一个 AI API”或“侧栏聊天”。

它做的是：

- 右侧独立 AI 镜像工作区
- 可选择外部 AI 目标，如豆包、DeepSeek、千问、WPS AI、文心一言、Kimi 等
- 通过桌面壳将目标网页 / 窗口映射进 FreeFlow
- 支持不同渲染方式与嵌入方式

代码证据：

- `electron/aiMirrorTargetManager.js`
- `electron/web/webContentsViewEmbed.js`
- `electron/win32/externalWindowEmbed.js`
- `public/src/runtime/workbenchRuntime.js`
- `public/src/engines/canvas2d-core/tutorial-system/tutorials/aiMirrorTutorial.js`

这意味着 FreeFlow 不是“白板 + AI 文本框”，而是：

> **把外部 AI 工作面板纳入自己的桌面工作台。**

这个点非常少见。

和现有主流白板相比，这属于显著差异化。

3.8 透明桌面壳 + window shape 同步，让画布和工作台变成桌面层

很多产品虽然也是 Electron，但只是常规桌面窗口。

FreeFlow 这边额外做了：

- 透明窗口
- always on top
- click-through 切换
- window shape 收集与同步
- 全局浮层与可交互区域管理

代码证据：

- `electron/main.js`
- `public/src/runtime/layout/windowShapeCollector.js`
- `public/src/runtime/layout/windowShapeSyncManager.js`
- `public/src/runtime/workbenchRuntime.js`

这个设计说明 FreeFlow 的目标不是“像网页一样开个应用”，而是：

- 更贴近桌面工作台
- 更贴近悬浮工作环境
- 更强调“工作台形态”

这在白板类产品里也不常见。

3.9 文件卡片不仅能挂文件，还能抽取文档内容进入系统

FreeFlow 对文件不是只做附件占位。

它已经有文件文本提取能力，能处理：

- `.docx`
- `.pptx`
- `.pdf`

代码证据：

- `src/backend/utils/fileTextExtractors.js`
- `src/backend/services/fileTextService.js`
- `src/backend/routes/persistenceRoutes.js`

这意味着文件在 FreeFlow 里不是单纯“一个图标”，而是未来可以成为：

- 可搜索内容
- 可理解内容
- 可再组织内容

这是向“资料工作台”方向走，而不只是画布摆放。

3.10 搜索不是只搜节点标题，而是搜正文、链接元数据、文件名、备忘录、标签

FreeFlow 当前搜索覆盖比很多画布工具更偏“内容检索”。

已看到的搜索字段包括：

- 标题
- 正文
- 链接 URL
- 链接元数据
- 备忘录
- 标签
- 文件名

代码证据：

- `public/src/engines/canvas2d-core/search/canvasSearchIndex.js`

这个点单独看不算惊世骇俗，但放在你的产品定位里很合理，因为它让画布更像知识工作区，而不是只做视觉摆放。

3.11 教程系统是产品化级别，不是一次性引导弹窗

FreeFlow 不只是有说明文档，而是把教程系统做成了独立运行时：

- 主界面教程
- 画布教程
- AI 镜像教程
- 教程板恢复 / 返回路径
- 快捷键速查

代码证据：

- `public/src/runtime/tutorial-system/`
- `public/src/engines/canvas2d-core/tutorial-system/`
- `src/backend/services/appStartupService.js`

这说明产品目标不是给少量高手用，而是想把较复杂的多模块桌面工作系统做成可上手产品。

这对于复杂工作台产品非常关键。

---

4. 哪些点是“组合创新”，不是单点绝对首创

下面这些点，单独看不一定全球独一份，但组合起来后很少见，且形成了 FreeFlow 自己的辨识度。

4.1 结构化导入 + 自有元素协议 + 办公导出

单独的：

- 富文本粘贴
- Markdown 支持
- Word 导出

都不是新鲜事。

但 FreeFlow 的不同在于：

- 输入端不是粗糙粘贴，而是 canonical 化
- 中间层不是黑箱富文本，而是元素协议
- 输出端不是截图导出，而是办公格式导出

这形成了完整的内容生命周期：

外部内容 -> FreeFlow 原生元素 -> FreeFlow 内部编辑组织 -> Office/文档流输出

这个链路是有辨识度的。

4.2 画布 + AI 对话 + AI 镜像 + 桌面壳

市面上常见的是：

- 白板工具有 AI 功能
- AI 工具有工作区

但 FreeFlow 的不同是：

- 左侧是画布工作区
- 右侧可以是 AI 对话或 AI 镜像
- 外部 AI 页面可以被工作台化
- 整体跑在桌面壳里

这不是“加个 AI 按钮”，而是重构工作台形态。

4.3 本地文件工作区 + 画布整理 + 文档办公流

很多白板工具更偏在线协作资产。
FreeFlow 更偏本地长期资料沉淀。

这种组合更接近：

- 项目资料整理
- 长期研究记录
- 内容编排后落 Word

所以它不是典型 brainstorming 白板，而是更偏生产资料台。

5. 哪些能力不能直接当“创新点”宣传

这部分要说清楚，不然对外口径会虚。

以下能力你有，但不能单独讲成创新：

- 无限画布
- 基本拖拽移动
- 多选
- 对齐吸附
- 文本 / 图片 / 图形 / 连线
- 缩放 / 平移
- 右键菜单
- 代码块 / 表格 / 数学公式
- 搜索
- 教程

这些都可以作为“能力完善度”来讲，但不能当核心创新。

真正能讲的，是它们和以下东西的结合：

- 结构化导入
- 办公导出
- AI 镜像
- 本地工作区
- 桌面壳

---

6. 与主流产品的横向对比

6.1 和 Miro 对比

Miro 的强项：

- 团队协作成熟
- 模板生态强
- 在线协作和大团队流程成熟
- 各类业务场景模板完善

FreeFlow 相比 Miro 的差异：

- FreeFlow 更本地、更桌面化，不是典型云协作平台
- FreeFlow 更强调结构化导入与格式转译，不只是白板承载
- FreeFlow 更强调 Word / 办公文档输出链路
- FreeFlow 有 AI 镜像工作区，这不是 Miro 核心形态
- FreeFlow 更像个人 / 小团队的生产型工作台，而不是模板驱动协作平台

一句话：

> Miro 强在协作平台化，FreeFlow 强在本地桌面工作流与办公流闭环。

6.2 和 FigJam 对比

FigJam 的强项：

- 轻量协作体验
- 组件、插件、widget、团队互动体验
- 适合会议、评审、头脑风暴

FreeFlow 相比 FigJam 的差异：

- FreeFlow 明显更偏“内容落地”而不是“会议过程”
- FreeFlow 的 Office 导出、结构化内容转译更深
- FreeFlow 不是以 widget 生态为核心，而是以内建办公流和桌面工作流为核心
- AI 镜像工作区是 FreeFlow 很独特的差异项

一句话：

> FigJam 更像协作白板，FreeFlow 更像桌面生产白板。

6.3 和 Excalidraw 对比

Excalidraw 的强项：

- 极简、零门槛
- 手绘风格辨识度强
- 开源、易集成
- 实时协作和导出简单直接

FreeFlow 相比 Excalidraw 的差异：

- FreeFlow 更重，更系统，不是轻量速绘工具
- FreeFlow 有结构化导入协议，不只是场景编辑器
- FreeFlow 有 Word 办公导出链路
- FreeFlow 有真实文件工作区和桌面壳
- FreeFlow 有 AI 镜像工作区
- FreeFlow 更强调多类型对象的办公流转换

一句话：

> Excalidraw 是轻量画图白板，FreeFlow 是重型桌面工作台白板。

6.4 和 AFFiNE 对比

AFFiNE 的强项：

- Docs / Whiteboard / Database 一体化
- 本地优先、自托管
- 一体化知识工作空间叙事清晰

FreeFlow 相比 AFFiNE 的差异：

- AFFiNE 强在 docs-whiteboard-database 的超融合
- FreeFlow 强在桌面工作台、外部 AI 镜像、办公文档流与本地工作区
- FreeFlow 对 Word 导出、复制直通、文件卡办公流更偏实战生产
- FreeFlow 更像“桌面工作台 + 办公白板”，而不是“通用知识 OS”

一句话：

> AFFiNE 更像知识系统平台，FreeFlow 更像办公生产工作台。

6.5 和 Obsidian Canvas 对比

Obsidian Canvas 的强项：

- 文件系统心智非常强
- Markdown / 笔记生态融合
- `.canvas` 开放格式
- 知识网络语义强

FreeFlow 相比 Obsidian Canvas 的差异：

- FreeFlow 更强调对象级结构化导入，不只是把文件/卡片摆到画布
- FreeFlow 更强调 Office 文档导出与办公复制直通
- FreeFlow 更强调桌面壳与 AI 镜像工作区
- FreeFlow 的工作区更偏“项目白板资料整理”，不只是知识卡片网络

一句话：

> Obsidian Canvas 更像知识可视化视图，FreeFlow 更像办公资料编排与输出工作台。

---

7. 横向对比表

维度	FreeFlow	Miro	FigJam	Excalidraw	AFFiNE	Obsidian Canvas
无限画布	有	有	有	有	有	有
多类型元素	强	强	中	中	强	中
外部内容结构化转译	**强**	中	中	弱	中	弱
粘贴后变成自有元素体系	**强**	中	中	弱	中	弱
Canonical 导入协议层	**有，且完整**	不明显	不明显	不明显	部分一体化	无明显独立协议层
办公导出导向	**强，偏 Word/Office**	中	弱	弱	中	弱
元素级多格式复制	**强**	中	中	中	中	弱
本地真实文件夹工作区	**强**	弱	弱	弱	中	**强**
AI 镜像工作区	**强，明显差异化**	弱	弱	无	弱	无
透明桌面壳 / shape 同步	**强，明显差异化**	无	无	无	弱	无
文档/PPT/PDF 文本抽取	**有**	弱	弱	弱	中	中
教程与工作台引导系统	强	强	强	中	中	中
主要产品气质	桌面办公工作台	在线协作平台	协作白板	轻量绘图白板	知识 OS	知识可视化

说明：
这里的“强 / 中 / 弱”是基于产品主轴判断，不是绝对是否支持某个单功能。

---

8. 我认为最值得对外强调的核心创新点

如果最后要提炼成最能打的 6 个点，我建议是下面这 6 个。

8.1 外部内容拖拽 / 粘贴后，不是临时显示，而是转成 FreeFlow 原生元素体系

这是最能体现产品底层价值的点。

可对外表述：

> 支持把外部文本、富文本、Markdown、代码、表格、公式、图片、文件拖入或粘贴到画布，并自动转译为可继续编辑、搜索、导出的 FreeFlow 原生对象。

8.2 结构化复制粘贴架构，让内容既能内部保真，又能对外兼容

可对外表述：

> 复制不是简单复制文本，而是同时维护内部结构化内容与外部兼容格式，兼顾 FreeFlow 内部保真与跨应用办公流。

8.3 画布内容可直接进入 Word / Office 流

可对外表述：

> 支持将画布整理结果直接生成正式 Word 文档，并保留标题、列表、表格、代码块、数学公式等结构，适合从灵感整理直接走向正式交付。

8.4 元素级办公直通复制

可对外表述：

> 单个富文本/表格/代码块对象就可以直接复制成 Word 富文本、PPT 富文本、Markdown、TSV 等格式，不必总是先导出整板。

8.5 AI 镜像工作区，把外部 AI 页面纳入同一桌面工作台

可对外表述：

> 不只是内置 AI 聊天，而是支持把外部 AI 页面或窗口直接映射进工作台，与画布并行协作。

8.6 本地文件夹式画布工作区

可对外表述：

> 画布不是封闭在 App 内部，而是直接映射真实本地文件夹，适合长期项目资料沉淀与个人工作流管理。

---

9. 如果从“架构创新”角度总结

如果不是面向用户宣传，而是面向技术/投资/架构讨论，我会这样概括：

9.1 FreeFlow 的核心不是渲染器，而是“内容协议层”

真正的技术中轴不是画出矩形，而是：

- 输入协议
- canonical 文档
- 元素桥接
- 对外兼容输出
- 导出 AST

这说明 FreeFlow 可以继续扩，不会被某一个编辑器或某一种富文本实现卡死。

9.2 FreeFlow 的产品边界比白板更宽

它的边界已经覆盖：

- 白板
- 资料整理
- 办公文档输出
- 本地项目工作区
- AI 协同工作面板

这就是它和“普通白板”的根本差异。

9.3 FreeFlow 的创新更像“工作流级创新”，不是单点组件创新

它最强的地方不是某个按钮，而是：

- 从输入到组织再到输出的整条链路
- 从桌面壳到画布再到 AI 的整条链路

这类创新更难抄，也更值得继续打磨。

---

10. 目前最可惜的地方

这部分不是说功能不行，而是指出“创新价值还没有被彻底放大”的地方。

10.1 很多创新点已经在代码里，但产品叙事还没完全收束

比如现在最值得讲的并不是“这是一个白板”，而是：

- 桌面工作白板
- 本地办公工作台
- 可把资料整理直接落成文档的画布系统

如果还按普通白板去讲，创新点会被稀释。

10.2 结构化导入很强，但对用户的可感知表达还不够

代码层已经很深，但用户未必第一时间知道：

- 为什么这里的粘贴和别人不一样
- 为什么贴进来后能继续编辑、搜索、导出

这个需要后续在 UI、引导、案例里强化。

10.3 AI 镜像非常特别，但需要更清晰的主场景包装

它现在从技术上已经很有差异化，但要让用户快速理解：

- 为什么不是普通 WebView
- 为什么它对工作流有价值
- 为什么和画布组合是强场景

这块未来值得继续打磨。

---

11. 最终判断

如果问：
FreeFlow 到底有哪些真正和其他画布产品不一样的地方？

我会给出最终结论：

第一层：最硬的不同

- 有一套独立的结构化导入 / 复制 / 回流协议架构
- 外部内容可以转译成 FreeFlow 原生元素体系，而不是仅做视觉保留
- 画布内容可直接进入 Word / Office 办公流
- 具备元素级多格式复制直通能力
- 具备 AI 镜像工作区
- 具备本地真实文件夹式画布工作区
- 具备透明桌面壳与窗口 shape 同步能力

第二层：组合起来后更独特

- 画布 + AI 助手 + AI 镜像 + 本地工作区 + 办公导出
- 导入结构化 + 编辑原生化 + 导出办公化

第三层：这决定了 FreeFlow 的真正定位

FreeFlow 最像的不是“另一个白板”，而是：

> **一个以画布为核心的本地桌面生产力工作台。**

---

12. 关键代码证据索引

为了后续继续扩写或对外整理，下面给一个最值得反复引用的代码索引。

结构化导入 / 复制协议

- `public/src/engines/canvas2d-core/import/README.md`
- `public/src/engines/canvas2d-core/import/runtime/createStructuredImportRuntime.js`
- `public/src/engines/canvas2d-core/import/canonical/canonicalDocument.js`
- `public/src/engines/canvas2d-core/import/protocols/canonicalFragmentCopy.js`
- `public/src/engines/canvas2d-core/import/protocols/externalCompatibilityOutput.js`
- `public/src/engines/canvas2d-core/import/gateway/pasteGateway.js`
- `public/src/engines/canvas2d-core/import/gateway/dragGateway.js`

画布原生元素桥接

- `public/src/engines/canvas2d-core/import/renderers/text/textElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/table/tableElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/code/codeBlockElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/math/mathElementBridge.js`
- `public/src/engines/canvas2d-core/import/renderers/file/fileCardElementBridge.js`

办公复制 / 导出

- `public/src/engines/canvas2d-core/export/copyExportProtocol.js`
- `public/src/engines/canvas2d-core/export/word/buildWordExportAst.js`
- `public/src/engines/canvas2d-core/export/word/buildWordExportPreviewModel.js`
- `electron/wordDocxCompiler.js`
- `electron/main.js`

本地工作区与启动恢复

- `public/src/engines/canvas2d-core/ui/BoardWorkspaceDialog.jsx`
- `public/src/engines/canvas2d-core/createCanvas2DEngine.js`
- `src/backend/services/canvasBoardService.js`
- `src/backend/services/appStartupService.js`

AI 镜像与桌面壳

- `electron/aiMirrorTargetManager.js`
- `electron/web/webContentsViewEmbed.js`
- `electron/win32/externalWindowEmbed.js`
- `public/src/runtime/workbenchRuntime.js`
- `public/src/runtime/layout/windowShapeSyncManager.js`

文件内容抽取与搜索

- `src/backend/utils/fileTextExtractors.js`
- `src/backend/services/fileTextService.js`
- `public/src/engines/canvas2d-core/search/canvasSearchIndex.js`

---

13. 外部参考资料

以下是本次横向对照中使用的主要官方资料：

- Miro Whiteboard / Help Center
  https://marketplace.miro.com/whiteboard/
  https://help.miro.com/hc/en-us/articles/360017557699-Web-whiteboard

- FigJam 官方与帮助文档
  https://www.figma.com/figjam/
  https://help.figma.com/hc/en-us/articles/1500004362321-Guide-to-FigJam

- Excalidraw 官方资料
  https://plus.excalidraw.com/
  https://docs.excalidraw.com/

- AFFiNE 官方资料
  https://affine.pro/
  https://docs.affine.pro/

- Obsidian Canvas 官方文档
  https://help.obsidian.md/plugins/canvas